

UNIDAD 4: Bases de Datos NoSQL

Bases de datos NoSQL - Conceptos

Las bases de datos **NoSQL** ("Not only SQL") están diseñadas para modelos de datos específicos, ofreciendo **esquemas flexibles, desarrollo iterativo, escalabilidad horizontal y un alto rendimiento** para grandes volúmenes de datos.

Principales diferencias con las bases de datos Relacionales:

- **No utilizan SQL** como lenguaje principal de consultas, ni estructuras fijas como tablas.
- **No suelen permitir operaciones JOIN**, solucionando esto mediante desnormalización o a nivel de aplicación.
- Poseen una **arquitectura distribuida**, operando sin cuellos de botella incluso en máquinas con pocos recursos.

Contraste entre los conceptos ACID, CAP y BASE

- **Teorema CAP:** En un sistema distribuido, solo se pueden garantizar **dos de las siguientes tres características: Consistencia** (todos los nodos leen la información más reciente), **Disponibilidad** (todos pueden leer lo disponible) y **Tolerancia a particiones** (el sistema funciona aunque se pierda conexión entre nodos).
- **Sistemas ACID (Relacionales):** Garantizan **Atomicidad** (la transacción se hace completa o se revierte), **Consistencia** (validaciones de reglas), **Aislamiento** (ejecución sin interferencia) y **Durabilidad** (protección ante fallos).
- **Sistemas BASE (NoSQL):** Sacrifican consistencia estricta para ganar **Alta Disponibilidad y Flexibilidad**. Ofrecen **Consistencia Eventual** (los nodos se sincronizan con el tiempo).

Historia y evolución

El acrónimo **NoSQL** ("Not only SQL") **surgió en 1998** como respuesta a las carencias de los sistemas relacionales (RDBMS). El verdadero auge llegó con la **Web 2.0 (Facebook, Twitter, YouTube)**, donde el contenido generado por los usuarios provocó un **crecimiento exponencial de datos**, haciendo inviable económica y técnicamente seguir escalando con bases de datos tradicionales.

Tipos de Bases de Datos NoSQL

1. **Key-value (Clave-Valor):** Almacenan datos en "diccionarios" o *contenedores* sin un esquema fijo. Ideales para consultas extremadamente rápidas de información básica. Ejemplos: **Redis, Riak**.
 2. **Document oriented (Orientadas a Documentos):** Guardan datos semi-estructurados usando documentos abstractos en formatos como **XML, JSON o BSON**. Ejemplos: **MongoDB, CouchDB**.
 3. **Graph (Grafos):** Almacenan información en **nodos (entidades)** y **aristas (relaciones)**. Superan a las RDBMS en la gestión y consulta veloz de **datos interconectados**. Ejemplo: **Neo4j**.
 4. **Column based (Wide Column Store):** Almacenan datos en familias de columnas en lugar de filas. Ideales para procesar **datos históricos, logs o eventos**. Ejemplos: **Cassandra, ScyllaDB**.
-

UNIDAD 5: Datos de Otras Fuentes

Datos Semiestructurados

Mientras los **datos estructurados** tienen formato y longitud definidos (tablas relacionales) y los **no estructurados** carecen de formato específico (PDF, video), los **datos semiestructurados** combinan ambos mundos. Se organizan mediante **metadatos o etiquetas**.

- **XML (Extensible Markup Language):** Usa **etiquetas** (ej. <libro>) para definir datos. Es clave para integrar flujos de información y automatizar **transacciones interempresariales**.
- **JSON:** Formato estándar basado en pares **clave:valor**. Es **ligero, rápido** y compatible con la mayoría de los lenguajes de programación, ideal para la web.

Bases de Datos Documentales

Son una variante clave de NoSQL que almacena información en **registros autónomos (JSON, XML o texto)**.

- **Ventajas:** Permiten consultar datos sin estructura definida, son **flexibles**, simplifican la adición de nuevos modelos, poseen **gran escalabilidad** para grandes volúmenes, y motores de búsqueda rápidos. Además, priorizan la **escritura rápida** sobre la consistencia absoluta.
- **Desventajas:** No tienen un **estándar definido ni utilizan SQL** nativo. No siempre garantizan propiedades ACID, tienen menor cantidad de recursos que las SQL, y sus **índices consumen mucha RAM**.

Estrategias para Integración de Datos Desde Múltiples Fuentes

Existen cinco metodologías principales para integrar datos:

1. **Integración manual:** Recopilación y visualización hecha a mano.
2. **Integración uniforme de acceso:** Muestra datos uniformes manteniéndolos en su fuente original.
3. **Integración de middleware:** Usa una capa intermedia para integrar sistemas heredados.
4. **Almacenamiento de datos común (Data Warehouse):** Se copia todo a una nueva base de datos centralizada.
5. **Integración basada en aplicaciones:** Software que extrae y armoniza la información.

UNIDAD 6: Otros Modelos de Bases de Datos

Bases de datos espaciales

Almacenan información vinculada al **espacio físico (2D o 3D)**, partes biológicas o diseños de ingeniería. A menudo inician como **imágenes digitales (bases de datos pictóricas)**.

- Capaces de consultar de forma conjunta **datos alfanuméricos y espaciales**.
- **Tipos de datos:** POINT (punto), LINE (línea), REGION (región).
- **Operaciones espaciales:** INSIDE (adentro), INTERSECTION (intersección), DISTANCE (distancia), integrables al lenguaje SQL.
- **Ejemplos:** PostGIS (el más potente/maduro), SpatialLite (portátil), Oracle, MySQL (muy limitado).

- **Usos:** Encontrar distancias en rutas, identificar inmuebles en un área delimitada, o buscar locales (ej. gasolineras) cercanos a una ubicación.

Bases de datos temporales y espacio/temporales

Las BD convencionales solo muestran una "instantánea" del presente. Las **bases de datos temporales gestionan datos que cambian en el tiempo**, permitiendo el **acceso eficiente a información histórica** (nóminas, historiales médicos).

- Los cambios **no sobrescriben** el estado, se ven como **adiciones**.
- Se extienden desde el modelo relacional agregando **atributos temporales** e implementando lenguajes como **TQUEL o SQL3**.

Taxonomía del Tiempo:

1. **Tiempo de transacción:** El momento en el que **el dato se almacena** en el sistema.
2. **Tiempo válido:** El momento en el que **el hecho es real o efectivo** (incluso puede ser en el futuro, como la vigencia de una tarjeta de crédito).
3. **Bitemporales:** Soportan ambas métricas simultáneamente.

Manejo por Tiempo de Transacción:

- Cada registro tiene un intervalo [**inicio, fin**). Si el objeto es actual, el fin marca "ahora".
- **El pasado no se puede modificar ("retroceso")**. Las actualizaciones solo aplican al presente.
- **La eliminación es 100% lógica, nunca física**. Al "borrar", simplemente se cierra el intervalo cambiando "ahora" por la fecha del borrado.

Tipos de consultas temporales:

- **Marca de tiempo (fracción):** "Empleados al 05/2022".
- **Range-timeslice:** "Empleados con ID entre 100-200 activos al 05/2022".
- **Intervalo (período):** "Empleados con ID entre 100-200 que trabajaron del 05/2021 al 06/2023".